

Objetivo

Implementar las funcionalidades avanzadas de la vista frontend del proyecto de solicitudes universitarias: paginación del servidor en `p-table`, un sistema global de notificaciones `Toast` invocable desde cualquier componente o interceptor, y optimización del rendimiento mediante `Lazy Loading` de rutas.

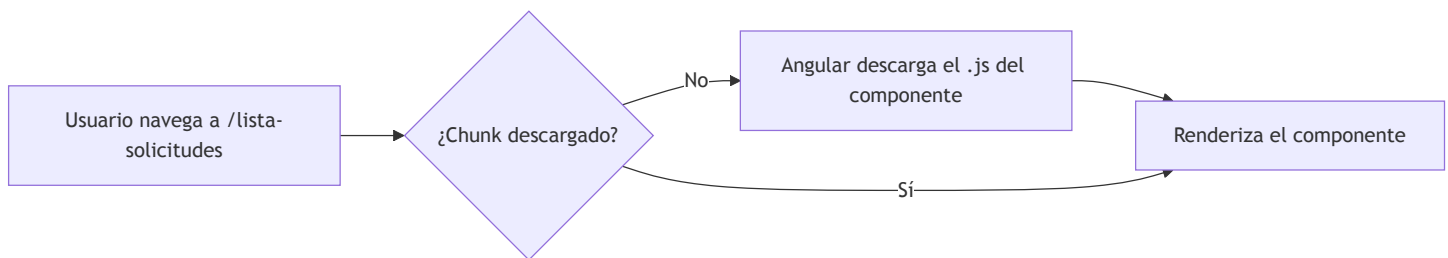
Conceptos Básicos Previos

- **Lazy Loading:** Carga diferida de componentes Angular bajo demanda al navegar a su ruta.
- **Server-Side Pagination:** El backend retorna solo un subconjunto de datos por página; el frontend envía `page` y `size` como parámetros.
- **Genéricos TypeScript (`<T>`):** Permiten crear tipos e interfaces reutilizables que funcionan con cualquier tipo de dato.
- **`effect()`** : Función de Angular 21 que ejecuta un efecto secundario cada vez que un signal cambia.
- **`toObservable()`** : Convierte un Signal en un Observable, parte del paquete `@angular/core/rxjs-interop`.
- **`MessageService`** : Servicio singleton de PrimeNG que alimenta el componente `p-toast` con mensajes.

Contextualización Teórica

Lazy Loading — Carga bajo demanda

Por defecto, Angular carga **todos** los componentes al inicio (*Eager Loading*). En proyectos grandes esto incrementa el tiempo de carga inicial. **Lazy Loading** difiere la descarga del código de un componente hasta que el usuario navega a su ruta:



Característica	Eager Loading	Lazy Loading
Cuándo se descarga	Al iniciar la app	Al navegar a la ruta
Tamaño del bundle inicial	Grande	Pequeño
Experiencia de usuario	Primera carga más lenta	Primera carga rápida
Implementación	<code>component: MiComponente</code>	<code>loadComponent: () => import(...).then(m => m.MiComponente)</code>

Impacto real: En el proyecto de solicitudes, los componentes `ListaSolicitudes` y `NuevaSolicitud` son pesados (incluyen PrimeNG). Cargarlos solo al navegar mejora el LCP (Largest Contentful Paint).

◆ Server-Side Pagination vs. Client-Side Pagination

Aspecto	Client-Side	Server-Side
¿Dónde pagina?	El navegador	El backend (Spring Boot)
Datos cargados	Todos a la vez	Solo la página actual
Parámetros de petición	Ninguno	<code>?page=0&size=10</code>
Ideal para	< 500 registros	Cualquier volumen
Respuesta del backend	<code>List<T></code>	<code>Page<T></code> (Spring) / <code>PageResponse<T></code>

Flujo de Server-Side Pagination:

```

Usuario cambia de página
    |
    ▼
onPageChange(event)           ← PrimeNG Paginator emite evento
page.set(event.page)          ← Signal actualizado
size.set(event.rows)          ← Signal actualizado
    |
    ▼
[toObservable + switchMap]    ← Angular 21 detecta el cambio
solicitudesService.listar(page, size) ← Petición HTTP con nuevos params
    |
    ▼
Backend retorna PageResponse<SolicitudResumen>
data.set(response.content)
totalElements.set(response.totalElements)

```

◆ Genéricos TypeScript — PageResponse<T>

Un **tipo genérico** (<T>) es un parámetro de tipo que se define al usar la interfaz, no al crearla. Permite que una misma estructura funcione con cualquier tipo de dato:

```

// ✅ Interfaz genérica – se define una vez, se usa para cualquier entidad
export interface PageResponse<T> {
  content: T[];           // array de elementos de tipo T
  totalElements: number;
  totalPages: number;
  size: number;
  number: number;         // página actual (0-indexed)
}

// Uso con SolicitudResumen:
const resp: PageResponse<SolicitudResumen> = await ...;
resp.content.forEach(s => console.log(s.codigo));

// El mismo tipo con cualquier otra entidad:
const usuarios: PageResponse<UsuarioDto> = await ...;

```

Sin genéricos, habría que crear `SolicitudPageResponse` , `UsuarioPageResponse` , etc. — violando el principio DRY.

◆ effect() en Angular 21

`effect()` ejecuta una función cada vez que **cualquier signal que lea dentro de ella** cambie. Es la forma de Angular 21 de reaccionar a cambios de signals para producir **efectos secundarios**:

```
// ✅ Uso correcto: efecto secundario (no modificar otro signal)
effect(() => {
  const msg = this.notificationService.message(); // lee el signal
  if (msg) {
    this.messageService.add({ ...msg, life: 4000 }); // efecto externo
  }
});
```

Reglas importantes de `effect()` :

1. **No modificar signals** dentro del efecto — causaría bucles infinitos. Usa `computed()` en su lugar.
2. Debe llamarse desde un **contexto de inyección** (constructor o con `injector`).
3. Es el patrón correcto para integrar signals con sistemas externos (DOM, librerías de terceros como PrimeNG Toast).

◆ **MessageService — Por qué es un proveedor**

`p-toast` de PrimeNG usa el patrón **Servicio + Componente**: el componente `<p-toast>` solo renderiza; el servicio `MessageService` decide qué mensajes mostrar. Este servicio **no tiene** `providedIn: 'root'` — debe registrarse manualmente en `app.config.ts` :

```
providers: [
  // ...
  MessageService // ← registro manual obligatorio para p-toast
]
```

Si no se registra, el error es: `NullInjectorError: No provider for MessageService` .

! **Precauciones y Recomendaciones**

1. PrimeNG instalado y configurado con preset `Aura` (Guía 17).
2. Guards `authGuard` y `publicGuard` implementados (Guía 18).
3. El backend debe soportar paginación: endpoint `GET /api/solicitudes?page=0&size=10` retornando `Page<SolicitudResumen>` de Spring.
4. En PrimeNG v21 se usan componentes directos (`Toast` , `Table` , `Paginator`) — no módulos (`ToastModule` , `TableModule`).

📦 **Artefactos Requeridos**

- Proyecto Angular con PrimeNG configurado (Guía 17) y Guards (Guía 18).
- Backend Spring Boot con endpoint paginado de solicitudes.

🏁 Evaluación o Resultado Esperado

Se espera que el estudiante:

1. Configure Lazy Loading en todas las rutas del proyecto.
2. Implemente un `NotificationService` global y lo integre con `p-toast`.
3. Consuma el endpoint paginado de solicitudes y lo muestre en `p-table` con paginador.
4. Verifique en DevTools que los chunks de Lazy Loading se descargan al navegar.

Procedimiento

PARTE 1: 🚀 Lazy Loading de Rutas

El Lazy Loading se implementa reemplazando `component: MiComponente` por `loadComponent: () => import(...)`.
Angular genera automáticamente un archivo `.js` separado ("chunk") para cada componente diferido.

1. Actualiza app.routes.ts con Lazy Loading

```
import { Routes }      from '@angular/router';
import { authGuard }   from './guards/auth.guard';
import { publicGuard } from './guards/public.guard';
import { rolesGuard }  from './guards/roles.guard';

export const routes: Routes = [
  // Ruta pública – publicGuard evita acceso si ya está autenticado
  {
    path: 'login',
    loadChildren: () => import('./componentes/login/login').then(m => m.Login),
    canActivate: [publicGuard]
  },

  // Rutas protegidas del proyecto de solicitudes
  {
    path: 'lista-solicitudes',
    loadChildren: () =>
      import('./componentes/lista-solicitudes/lista-solicitudes')
        .then(m => m.ListaSolicitudes),
    canActivate: [authGuard]
  },
  {
    path: 'nueva-solicitud',
    loadChildren: () =>
      import('./componentes/nueva-solicitud/nueva-solicitud')
        .then(m => m.NuevaSolicitud),
    canActivate: [authGuard]
  },

  // Página de acceso no autorizado (sin guard)
  {
    path: 'unauthorized',
    loadChildren: () =>
      import('./componentes/unauthorized/unauthorized')
        .then(m => m.Unauthorized)
  },

  // Ruta para administradores (si aplica en tu proyecto)
  {
    path: 'admin',
    loadChildren: () =>
      import('./componentes/admin/admin').then(m => m.Admin),
    canActivate: [rolesGuard],
    data: { expectedRoles: ['ADMIN'] }
  },

  { path: '', redirectTo: '/login', pathMatch: 'full' },
  { path: '**', redirectTo: '/' }
];
```

loadComponent recibe una función de importación dinámica. El `import()` es una característica de ES2020 que retorna una Promesa. El `.then(m => m.NombreClase)` extrae el componente del módulo importado.

2. Verificación en DevTools

```
npm start
```

1. Abre F12 → pestaña **Network** → filtra por **JS**.
2. Recarga la app — observa el bundle inicial (pequeño).
3. Navega a `/lista-solicitudes` — debe aparecer un nuevo archivo `.js` descargado.
4. Navega a `/nueva-solicitud` — otro chunk adicional.

PARTE 2: 📄 DTO de Paginación Genérico

1. Crea la interfaz `PageResponse<T>` :

```
ng generate interface dto/page-response
```

2. Modifica `src/app/dto/page-response.ts` :

```
/**
 * Interfaz genérica para respuestas paginadas del backend.
 * Equivale a org.springframework.data.domain.Page<T> de Spring Boot.
 *
 * @template T - Tipo del elemento en el listado (ej: SolicitudResumen, UsuarioDto)
 */
export interface PageResponse<T> {
  content: T[]; // elementos de la página actual
  totalElements: number; // total de registros en la BD
  totalPages: number; // total de páginas
  size: number; // tamaño de página
  number: number; // índice de página actual (0-indexed)
}
```

¿Por qué genérico? `PageResponse<SolicitudResumen>` y `PageResponse<UsuarioDto>` reusan la misma interfaz. Si necesitas paginación para una nueva entidad, no creas una nueva interfaz — simplemente usas `PageResponse<NuevaEntidad>`.

PARTE 3: 🛎 Servicio Global de Notificaciones

3.1 Crea el servicio:

```
ng generate service servicios/notification --skip-tests
```

3.2 Implementa src/app/servicios/notification.ts :

```
import { Injectable, signal } from '@angular/core';

export interface ToastMessage {
  severity: 'success' | 'info' | 'warn' | 'error';
  summary: string;
  detail: string;
}

/**
 * Servicio singleton de notificaciones globales.
 * Cualquier componente, servicio o interceptor puede invocarlo.
 * App.ts escucha el signal y lo publica en p-toast.
 *
 * Nota: el nombre NotificationService evita el conflicto con window.Notification del navegador.
 */
@Injectable({ providedIn: 'root' })
export class NotificationService {
  // Signal con el último mensaje – App.ts reacciona con effect()
  message = signal<ToastMessage | null>(null);

  success(summary: string, detail: string) {
    this.message.set({ severity: 'success', summary, detail });
  }

  error(summary: string, detail: string) {
    this.message.set({ severity: 'error', summary, detail });
  }

  warn(summary: string, detail: string) {
    this.message.set({ severity: 'warn', summary, detail });
  }

  info(summary: string, detail: string) {
    this.message.set({ severity: 'info', summary, detail });
  }
}
```

3.3 Integra p-toast en app.ts :

El componente `p-toast` debe estar en el componente raíz para ser visible en todas las páginas.


```

import { Component, inject, effect } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Header } from '../componentes/header/header';
import { Footer } from '../componentes/footer/footer';
import { Toast } from 'primeng/toast'; // ✅ PrimeNG v21 standalone
import { MessageService } from 'primeng/api';
import { NotificationService } from '../servicios/notification';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Header, Footer, Toast],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  private notificationService = inject(NotificationService);
  private messageService = inject(MessageService);

  constructor() {
    // ✅ effect(): cada vez que notificationService.message() cambie,
    // se envía el mensaje al MessageService de PrimeNG que alimenta p-toast
    effect(() => {
      const msg = this.notificationService.message();
      if (msg) {
        this.messageService.add({
          severity: msg.severity,
          summary: msg.summary,
          detail: msg.detail,
          life: 4000
        });
      }
    });
  }
}

```

¿Por qué effect() aquí? Necesitamos un efecto secundario (llamar a `messageService.add()`) cada vez que el signal `message` cambie. `effect()` es el mecanismo correcto para integrar signals con sistemas externos. No se modifica ningún signal dentro del efecto — solo se llama a un método externo.

3.4 Actualiza app.html :

```
<!-- p-toast debe estar en el root para ser visible globalmente -->
<p-toast position="top-right"></p-toast>

<app-header></app-header>

<router-outlet></router-outlet>

@defer (on viewport) {
  <app-footer></app-footer>
}
```

3.5 Registra MessageService en app.config.ts :

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { providePrimeNG } from 'primeng/config';
import { Aura } from 'primeng/themes/aura'; // ✅ v21
import { MessageService } from 'primeng/api';
import { routes } from './app.routes';
import { authInterceptor } from './interceptores/auth.interceptor'; // ✅ nombre correcto

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
    provideAnimationsAsync(),
    MessageService, // ← obligatorio para p-toast
    providePrimeNG({ theme: { preset: Aura, options: { darkModeSelector: '.app-dark' } } })
  ]
};
```

3.6 Actualiza el interceptor para usar NotificationService :

```
// src/app/interceptores/auth.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';
import { AuthService } from '../servicios/auth';
import { NotificationService } from '../servicios/notification';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const router = inject(Router);
  const notificationService = inject(NotificationService);

  if (!authService.isAuthenticated()) {
    return next(req);
  }

  const authReq = req.clone({
    setHeaders: { Authorization: `Bearer ${authService.getToken()}` }
  });

  return next(authReq).pipe(
    catchError(error => {
      if (error.status === 401) {
        authService.logout();
        router.navigate(['/login']);
        notificationService.warn('Sesión expirada', 'Por favor inicie sesión nuevamente.');
      } else if (error.status === 403) {
        notificationService.error('Acceso denegado', 'No tiene permisos para esta acción.');
      } else if (error.status === 0) {
        notificationService.error('Sin conexión', 'No se pudo conectar con el servidor.');
      }
      return throwError(() => error);
    })
  );
};
```

3.7 Usa el servicio de notificaciones desde cualquier componente:

```
import { Component, inject } from '@angular/core';
import { NotificationService } from '../servicios/notification';

export class NuevaSolicitud {
  private notificationService = inject(NotificationService);

  guardar() {
    // ... lógica de guardado
    this.notificationService.success('Guardado', 'La solicitud fue registrada correctamente.');
```

PARTE 4: Lista de Solicitudes con Paginación del Servidor

Aplicamos todo lo anterior al componente `ListaSolicitudes` del proyecto, usando `p-table` con paginación server-side.

4.1 Añade el método `listar paginado` en `SolicitudesService` :

```
// src/app/servicios/solicitudes.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';
import { SolicitudResumen } from '../modelos/solicitudes';
import { PageResponse } from '../dto/page-response';

@Injectable({ providedIn: 'root' })
export class SolicitudesService {
  private readonly API = 'http://localhost:8080/api/solicitudes';
  private http = inject(HttpClient);

  /** Carga todas las solicitudes sin paginación (p.ej. para el listado simple de Guía 16) */
  listar(): Observable<SolicitudResumen[]> {
    return this.http.get<SolicitudResumen[]>(this.API);
  }

  /** Carga solicitudes paginadas – para p-table con paginación del servidor */
  listarPaginado(page: number, size: number): Observable<PageResponse<SolicitudResumen>> {
    const params = new HttpParams()
      .set('page', page)
      .set('size', size);
    return this.http.get<PageResponse<SolicitudResumen>>(this.API, { params });
  }
}
```

4.2 Actualiza lista-solicitudes.ts — patrón Angular 21 reactivo:

```
import { Component, computed, inject, signal } from '@angular/core';
import { toSignal, toObservable } from '@angular/core/rxjs-interop';
import { RouterLink } from '@angular/router';
import { SlicePipe } from '@angular/common';
import { switchMap, catchError, of } from 'rxjs';

// PrimeNG v21 — componentes standalone
import { Table } from 'primeng/table';
import { Paginator, PaginatorState } from 'primeng/paginator';
import { ProgressSpinner } from 'primeng/progressspinner';
import { Tag } from 'primeng/tag';

import { SolicitudesService } from '../../servicios/solicitudes';
import { NotificationService } from '../../servicios/notification';
import { SolicitudResumen } from '../../modelos/solicitudes';
import { PageResponse } from '../../dto/page-response';

const EMPTY_PAGE: PageResponse<SolicitudResumen> = {
  content: [], totalElements: 0, totalPages: 0, size: 5, number: 0
};

@Component({
  selector: 'app-lista-solicitudes',
  imports: [RouterLink, SlicePipe, Table, Paginator, ProgressSpinner, Tag],
  templateUrl: './lista-solicitudes.html',
  styleUrls: ['./lista-solicitudes.css']
})
export class ListaSolicitudes {
  private solicitudesService = inject(SolicitudesService);
  private notificationService = inject(NotificationService);

  // ✅ Signals que controlan la paginación
  page = signal(0);
  size = signal(5);

  // ✅ computed() une page y size en un objeto — cualquier cambio activa el switchMap
  private params = computed(() => ({ page: this.page(), size: this.size() }));

  // ✅ Patrón Angular 21 de paginación reactiva:
  //   toObservable(params) → Observable que emite cuando page o size cambian
  //   switchMap           → cancela la petición anterior y lanza una nueva
  //   toSignal            → convierte el resultado en un Signal (sin subscribe())
  paginatedData = toSignal(
    toObservable(this.params).pipe(
      switchMap(p =>
        this.solicitudesService.listarPaginado(p.page, p.size).pipe(
          catchError(() => {
            this.notificationService.error('Error', 'No se pudo cargar el listado.');
```

```

    )
  )
),
{ initialValue: EMPTY_PAGE }
);

// ✅ computed() derivados del signal de datos
solicitudes = computed(() => this.paginatedData().content);
totalElements = computed(() => this.paginatedData().totalElements);
cargando = computed(() => this.solicitudes().length === 0 && this.totalElements() === 0);

onPageChange(event: PaginatorState): void {
  this.page.set(event.page ?? 0);
  this.size.set(event.rows ?? 5);
  // ✅ No hay que llamar a ninguna función – el cambio en los signals
  // activa automáticamente toObservable → switchMap → nueva petición
}

tagSeveridad(estado: string): 'success' | 'info' | 'warn' | 'danger' | 'secondary' | 'contrast' {
  const mapa: Record<string, 'success' | 'info' | 'warn' | 'danger' | 'secondary' | 'contrast'> = {
    REGISTRADA: 'secondary', CLASIFICADA: 'info',
    EN_ATENCION: 'contrast', ATENDIDA: 'success',
    CERRADA: 'secondary', CANCELADA: 'danger',
  };
  return mapa[estado] ?? 'secondary';
}

tagPrioridad(prioridad: string): 'success' | 'warn' | 'danger' {
  return ({ BAJA: 'success', MEDIA: 'warn', ALTA: 'danger' } as Record<string, 'success' | 'warn' | 'da
    [prioridad] ?? 'success';
}
}

```

¿Por qué **switchMap**? Si el usuario cambia de página rápidamente, **switchMap** **cancela** la petición HTTP anterior y envía solo la más reciente. **mergeMap** enviaría todas en paralelo — lo cual generaría condiciones de carrera con datos desactualizados.

4.3 Actualiza lista-solicitudes.html :

```
<div class="container mt-4">

  <div class="d-flex justify-content-between align-items-center mb-4">
    <h1 id="titulo-listado" class="h3 mb-0">
      <i class="pi pi-list-check me-2"></i>Mis Solicitudes
    </h1>
    <a routerLink="/nueva-solicitud" class="btn btn-primary">
      <i class="pi pi-plus me-1"></i>Nueva Solicitud
    </a>
  </div>

  @if (cargando()) {
    <div class="d-flex justify-content-center my-5">
      <p-progressSpinner strokeWidth="4" />
    </div>
  } @else {
    <p-table
      [value]="solicitudes()"
      [tableStyle]="{ 'min-width': '50rem' }"
      stripedRows>

      <ng-template #header>
        <tr>
          <th>Código</th>
          <th>Tipo</th>
          <th>Estado</th>
          <th>Prioridad</th>
          <th>Fecha</th>
        </tr>
      </ng-template>

      <ng-template #body let-solicitud>
        <tr>
          <td><strong>{{ solicitud.codigo }}</strong></td>
          <td>{{ solicitud.tipo }}</td>
          <td>
            <p-tag [value]="solicitud.estado"
              [severity]="tagSeveridad(solicitud.estado)" />
          </td>
          <td>
            <p-tag [value]="solicitud.prioridad"
              [severity]="tagPrioridad(solicitud.prioridad)"
              icon="pi pi-arrow-up" />
          </td>
          <td>{{ solicitud.fechaCreacion | slice:0:10 }}</td>
        </tr>
      </ng-template>

      <ng-template #emptymessage>
        <tr>
```

```

        <td colspan="5" class="text-center py-4">
            <i class="pi pi-inbox" style="font-size: 2rem"></i>
            <p class="mt-2 text-muted">No hay solicitudes registradas.</p>
        </td>
    </tr>
</ng-template>

</p-table>

<!-- Paginador del servidor -->
<p-paginator
    [rows]="size()"
    [totalRecords]="totalElements()"
    [rowsPerPageOptions]="[5, 10, 20]"
    (onPageChange)="onPageChange($event)">
</p-paginator>
}

</div>
```

ng-template #header / #body / #emptymessage : En PrimeNG v17+, `p-table` usa **template reference variables** (con `#`) en lugar de `pTemplate` para definir las zonas de la tabla. `#header` define la fila de encabezado, `#body` `let-solicitud` itera las filas con la variable `solicitud`, y `#emptymessage` se muestra cuando no hay datos.

stripedRows : Atributo boolean de `p-table` que alterna el fondo de las filas (zebra stripes) para mejorar la legibilidad.

4.4 Verificación completa:

```
npm start
```

Prueba	Acción	Resultado esperado
Paginación	Cambia de página en el paginador	La tabla carga los datos de la nueva página
Lazy Loading	DevTools → Network → JS	Nuevo chunk <code>.js</code> al navegar por primera vez
Toast de error	Detén el backend y recarga	Toast rojo: "Sin conexión"
Toast 401	Modifica el token en localStorage	Toast naranja: "Sesión expirada" + redirect login
Autorización	Revisa request en DevTools Network	Header <code>Authorization: Bearer ...</code> presente

PARTE 5: 🎯 Avance del Proyecto Final

1. Aplica `loadComponent` (Lazy Loading) a **todas** las rutas de tu proyecto.
2. Verifica que el `NotificationService` es invocado desde el componente `NuevaSolicitud` al guardar y al recibir errores.
3. Confirma que el `authInterceptor` muestra notificaciones Toast para errores 401, 403 y sin conexión.

4. Valida en DevTools que las peticiones paginadas incluyen los parámetros `page` y `size` .

Control de Versiones

```
git add .
git commit -m "feat: lazy loading, NotificationService y p-table paginada"
git push origin main
```

Solución de Problemas

- **NullInjectorError: No provider for MessageService :**
 - Añade `MessageService` al arreglo `providers` en `app.config.ts` .
- **El Toast no aparece aunque llamo a `notificationService.success()` :**
 - Verifica que `<p-toast>` está en `app.html` (no en un componente hijo).
 - Verifica que `effect()` está en el constructor de `App` .
 - Verifica que `Toast` (no `ToastModule`) está en `imports` del `App` .
- **`p-table` no muestra datos aunque `solicitudes()` tiene elementos:**
 - Verifica que `[value]="solicitudes()"` incluye los paréntesis del signal.
 - Verifica que `Table` está en `imports` del componente.
- **`switchMap` importado pero TypeScript da error:**
 - Asegúrate de: `import { switchMap, catchError, of } from 'rxjs'` .
- **El paginador no actualiza la tabla:**
 - Verifica que `onPageChange` llama a `this.page.set(...)` y `this.size.set(...)` .
 - El `computed` `params` detectará el cambio y `toObservable` emitirá automáticamente.

Para la Próxima Clase

1. Investiga `p-dialog` de PrimeNG para crear formularios modales (edición en línea sin navegar):
 - Referencia: [PrimeNG Dialog](#)
2. Explora **búsqueda en tiempo real** con `debounceTime` de RxJS + signals:
 - Referencia: [RxJS debounceTime](#)
3. Investiga el operador `startWith` de RxJS para mostrar un estado de carga inicial:
 - Referencia: [RxJS startWith](#)



Preguntas para Reflexión

- ¿Por qué se usa `switchMap` en lugar de `mergeMap` para paginación? ¿Qué ocurriría con `mergeMap` si el usuario hace clic rápido en las páginas?
- ¿Qué ventaja tiene `toObservable(params) + switchMap` sobre llamar `subscribe()` manualmente en `onPageChange()` ?
- ¿Por qué `PageResponse<T>` usa un genérico en lugar de ser una interfaz específica para `SolicitudResumen` ?
- ¿Cuándo es correcto usar `effect()` en Angular 21? ¿Qué pasa si modificas un signal dentro de un `effect()` ?
- ¿Qué diferencia hay entre `p-table` con Lazy Loading de rutas y `@defer` de Angular? ¿Cuándo usar cada uno?



Referencias Bibliográficas

- [Angular Lazy Loading](#)
- [Angular effect\(\)](#)
- [Angular toObservable\(\)](#)
- [PrimeNG Table](#)
- [PrimeNG Paginator](#)
- [PrimeNG Toast](#)
- [RxJS switchMap](#)
- [TypeScript Generics](#)